SQL Connection and File handling

24 May 2025 09:54

```
#Sql Connection or Database Connection or Backend Connection
'''
What is SQL Connection :
   it is used to establish the connection between pythonfile and data base
Why SqL:
   how to connect with python file
#whenever we should have a communication between python file and database
   file we are going to make use of "connect()"
#Step1: to use connect() first we should import sqlite3(Module).
   import sqlite3
#Step2: after importing sqlite3 we can make use of connect().
   syntax: variablename=sqlite3.connect('databasefile.db')
#Step3:  just to enter the database and to perform the operations we will be making use
            of "cursor()".
   Syntax: variablename1=variablename.cursor()
#Step4: now to execute Queries will be making use of inbuilt function "execute()"
   *execute() will take "string input",where we are going to mention all the
   queries which we wants to execute.
         Syntax: variablename1.execute('Query')
#step 5:after executeing Query we did some Modifications ,to save those Modifications
         we are going to use inbuilt function called "commit()"
         syntax: variablename.commit()
# Step 6: After performing all the operations we should have to remove the
         esblish connection between pythonfile and database file.we will make use
         of inbuilt function called "close()"
         Syntax:variablename.close()
#Table creation:
Syntax: create table tablename(column1 datatype constraint,
                                    column2 datatype constraint,
                                    column3 datatype constraint,
                                    column n datatype constraint)
Note: constraint is optional,for char() no need to give size.
#to insert values:
syntax: insert into tablename values(value1,value2,value3....valuen)
#To Extract data:
   syntax: select * from Tablename---->all records/details
           select columnname from tablename--->all records of one column
           select * from tablename where condition.
```

```
import sqlite3
a=sqlite3.connect(r"C:\Users\USER\OneDrive\Desktop\Database.db")
b=a.cursor()
b.execute('create table Amazon(prod_id num,prod_name char,prod_cose num)')
b.execute("insert into  Amazon values(1,'iphone',100000)")
b.execute("insert into  Amazon values(2,'laptop',50000)")
b.execute("insert into Amazon values(3,'book',10000)")
a.commit()
res=b.execute("select * from  Amazon")
print(list(res))
'''
import sqlite3
a=sqlite3.connect(r'C:\Users\USER\OneDrive\Desktop\Student.db')
b=a.cursor()
b.execute('create table Student(s_id num,s_name char,s_age num)')
b.execute("insert into  Student values(101,'parsvi',10)")
b.execute("insert into  Student values(2,'Davya',15)")
b.execute("insert into Student values(3,'Pavithra',15)")
a.commit()
res=b.execute("select * from  Student")
print(list(res))

#Renaming the db file
import os
dbfile='Flipkart.db'
newdbfile='Myntra'
if os.path.exists(dbfile): #os.remove(dbfile)
      os.rename(dbfile,newdbfile)
      print('database renamed successfully')
 else: print('databse file is not found')

#Deleting the db file
 import os
 dbfile='Amazon1.db'
if os.path.exists(dbfile):
     os.remove(dbfile)
     print('database deleted successfully')
 else: print('databse file is not found')
```

Note :Do above programs in visual studio

#FileHandling: '''

What do you mean by file?

It is a container which is used to store the data . What type of data ,can we store inside the file? Any type of data we can store. Example:txt file,image files,audio files,video files,word, document,excel file etc...

What do you mean by file handling?

Whenever we consider the any file three things we can perform over here.

1.We can read the contant from the file.

2.We can write the contant into the file.

3.We can do any updates inside the file.

Definition: It is a phenomenon of reading the data from the file or writting the data into the file.

To get the accessibility of any file we will be using inbuild function called "open()" open() function having two arguments 1.location or file name 2.Mode Of Operation

Syntax for open function: variable_name=open('location file/file.exe(filename)','Mode Of Operation')

Mode Of Operation got classified into 3 types:

1.Write(w): w(write only),

w+(write+read),

wb(It will covert into binarydata for security purpose),

wb+(It used to write and read binary data)(r):

2.r(read only),

r+(read+write),

rb(It is used to read the binary data),

rb+(It is used to first read and 2nd write binary data)

3.Append(a):

a(It will add data in existed data and add at the end),a+(first we can add and second we can read the data.),

a+

ab(used to add binary data.) ,

ab+(first add and then read the binary data.)

By using python we are going to work with respect to two files
1.Text File 2.CSV file

1.Text File: Write:
*It is mode of operation which is used to write the data into the file.
*After getting accessibilty if ever we want to write the data ,we will using inbuilt functions

1.write()/(write function):It is used to write single line data into file.
Syntax:variable_name.write('data')

2.writelines():It is used to write multiple lines of data at a time into file.
Syntax:variable_name.writelines(['data1(line1)','data2'....'datan'])

Note:After doing write and read we need to close the file when we do not close what happens?
It will not update anything or otherwise output will not display.
Syntax:variable_name.close()

```
#writting the data into the file
fo=open('FILE_1.txt','w')
fo.write('Good Afternoon')
fo.writelines(['Good Afternoon ','Good Morning ','Good Night'])
fo.writelines(['Good Afternoon\n','Good Morning\n','Good Night'])
fo.close()
```

'write(w):only write

*opens the file for writing .for existing file data is truncated and overwritten.Cursor is placed at the beginning of the file.
*Creats the file if the file doesn't exist.
*Writing to file:

1.write():insert the data in a single line
Syntax: variablename.write('single line data')
2.writelines():it is used to insert multiple strings at a time.
Syntax: variablename.writelines(['data1','data2','dat3'...'datan'])

#Working with Respect to Text file

#close():

1.it creates a connection between your program and the file.
2.Allocates memory resources to handle reading /writing into the file.
3.Places a file pointer(cursor) to track where your reading/writing.

```
file1=open('TEXTFILE.txt','w')
file1.write('hello students Good morning')
file1.writelines([' hi',' hello',' bye'])
file1.close()
```

#with statement:
* no need to call file1.close().with statement will take care of it.
*it ensures proper acquisition and release od resources.
Syntax:

```
   with open('file.exe/path','mode of operation') as variablename:
   <----> -----instructions-----
with open('TEXTFILE.txt','w') as file1:
   file1.write('hello students Good morning')
   file1.writelines([' hi',' hello',' bye'])
```

#read(r):
*opens the file for reading,cursor is pointed at the beginning of the file ,if the file doesn't exist,raise FileNotFoundError.
*which is used to read the data from the file or access or extract data from the file.

Syntaxes:

1.read(): it will print entire data present in the file
syntax: var=variablename.read()
2.realine():it will print first line from the text file
syntax: var=variablename.readline()
3.radlines: it will store all the data iside the list
syntax: var=variablename.readlines()

```
with open('TEXTFILE.txt','r') as file1:
   #res=file1.read()
   #res=file1.readline()
   #res=file1.readlines()
   print(file1.readlines())
```

```
with open(r'C:\Users\USER\Desktop\TEXTFILE1.txt','w') as file1:
    file1.write('this is example with path')

with open(r'C:\Users\USER\Desktop\TEXTFILE1.txt','r') as file1:
    print(file1.read())
```

#append(a) :
opens the file for writing .the file is created if it doenot exist.
cursor is placed at the end of the file.
the data being written will be inserted at the end,after the existing data.

```
with open('TEXTFILE2.txt','a') as file1:
    file1.write(' this is example of append mode')

'''with open(r'C:\Users\USER\Desktop\TEXTFILE1.txt','a') as file1:
    file1.write(' this is example of append mode')
```

W+(write and read):
open the file for writing and reading,for exsiting file data truncated
 and overwritten.cursor is placed at the beginning of the file.
*it will create a file if doesnot exist.

```
with open('SAMPLE.txt','w+') as file2:
    file2.write('Fresh content')
    file2.write('new content')
    file2.seek(0)
    print(file2.read())
file = open("filename.txt", "w+")
```

Key Characteristics of "w+":

Creates a new file if it does not exist.
Overwrites the file (truncates it to zero length) if it already exists.
Allows both reading and writing.

Scenario 1: File Exists

Behavior:
The file is cleared immediately when opened with "w+".
Then you can write and read from it, but:
After writing, you need to move the cursor back to the beginning using seek(0) to read what you wrote.

Example:

```
# Let's say "file.txt" already exists and has some content
f = open("file.txt", "w+")
f.write("Hello")
f.seek(0)          # Move to beginning
print(f.read())     # Output: Hello
```

Important Notes:
Any old content is lost.
If you read before writing, it returns an empty string because the file is empty after being truncated.

Scenario 2: File Does Not Exist

Behavior:

Python will create the file automatically.
File is empty initially.
You can write to it and read from it as long as you manage the cursor position.
Example:

```
python
Copy code
f = open("newfile.txt", "w+")
print(f.read())     # Output: '' (empty string, since file was just created)
f.write("Test")
f.seek(0)
print(f.read())     # Output: Test
```

seek():
it is used to make your cursor to point to the perticular index where you want.
syntax: variablename.seek(indexposition)

Note:
w+:the priority will be given to write mode by default the cursor will be blinking at the last position.
Reading immediately after opening in "w+" will not return old content — it will return ''
because the file was truncated (even if it existed).

#read and write(r+):
"r+" Mode Explained
file = open("filename.txt", "r+")

Key Characteristics of "r+":
Opens the file for both reading and writing.
Does NOT create the file if it doesn't exist.
Does NOT truncate the file (it keeps the existing content).
The file pointer starts at the beginning of the file.

Scenario 1: File Exists
Behavior:
You can read and write freely.
Writing overwrites content starting from the current cursor position.
To append, you need to move the cursor to the end using seek().
Example:

```
# Suppose file.txt contains: "Hello World"
f = open("file.txt", "r+")
print(f.read())      # Output: Hello World
f.seek(0)            # Move to beginning
f.write("Hi")        # Overwrites 'Hello' with 'Hi'
f.seek(0)
print(f.read())      # Output: Hi llo World
```

Important Notes:
It does not delete the old content.
It overwrites in-place starting from the current position.

Scenario 2: File Does Not Exist

Behavior:
Python raises a FileNotFoundError.
like "w+", it does not create the file.

```
        Example:
        f = open("nofile.txt", "r+")
        # Raises: FileNotFoundError
        Example:
        with open('SAMPLE1.txt','r+') as file2:
          print(file2.read())
          file2.write('this is not example of write mode')
'''


with open('SAMPLE1.txt','r+') as file2:
  print(file2.read())
  file2.write('this is not example of write mode')
   #print(file2)


  #file2.seek(0)
  #print(file2.read())
#a+--->append+read
with open('append1.txt','a+')as file:

  file.seek(0)
  print(file.read())
  #file.write('this is the example of append file')
```

Note:
  w+:write and read(on new File)
  read():if you do read operation first it is displaying empty.

  write():if you perform write operation after read().it will
  write the data into file.but we are not get any output in idle.why
  means first reading next writing...that mean before writing the data
  there nothing.after writing the data the cursor blinking at the last
  if we want to read that data in idle..there we have to use seek().
  but.first write operation,then seek(),then read()...then only we will

get output.

*in w+ the data overwritten.
*new file created if the file doen't exist.

a+:append and read(newly created file)

read():if you do read operation first it is displaying empty.]

write():if you perform write operation after read().it will
write the data into file.but we are not get any output in idle.why
means first reading next writing...that mean before writing the data
there nothing.after writing the data the cursor blinking at the last
if we want to read that data in idle..there we have to use seek().
but.first write operation,then seek(),then read()...then only we will
get output.
*in a+ the data added with existing data
*new file created if the file doen't exist.

```
with open('write.txt','w+')as file:
  file.write('this is the example of xyz file')
  file.seek(0)
  res=file.read()
  print(res)

#w+ on existed file
with open('SAMPLE1.txt','w+')as file:
  file.write('hi hello how are you')
  file.seek(0)
  res=file.read()
  print(res)
#a+ on existed file
with open('SAMPLE1.txt','a+')as file:
  file.write('hiiiiiiii')
  file.seek(0)
  res=file.read()
  print(res)
```

```
#r+ on existed file-->read+write
with open('SAMPLE1.txt','r+')as file:
  file.write('ppppppppppppppppppppp')--->wirte mode
  -->it is overwrittens old data with new data at the begining
  the cursor blonking at last 'l'.if you perform read() it is going to print
  the data after cursor.
  file.seek(0)
  print(file.read())
  file.write('morning batch ')--->append mode
```

Notes:i should add some point related r+

```
#wap program to extract number of words present in a given text
file(SAMPLE1)

with open('SAMPLE1.txt','r+')as file:
  res=file.read()
  words=res.split()
  print(len(words))#27,236characters

#internal Process:
len('pppppppppppppppppppppbyee....bye...write file welcom appned
filebyee....bye...write file welcom appned filethis the read modehiiiiiiiihiiiiiiiithis
the read mode this the read mode morning batch morning batch morning batch
morning batch ')

res='pppppppppppppppppppppbyee....bye...write file welcom appned
filebyee....bye...write file welcom appned filethis the read modehiiiiiiiihiiiiiiiithis
the read mode this the read mode morning batch morning batch morning batch
morning batch '
res.split()
['pppppppppppppppppppppbyee....bye...write', 'file', 'welcom', 'appned',
'filebyee....bye...write', 'file', 'welcom', 'appned', 'filethis', 'the', 'read',
'modehiiiiiiiihiiiiiiiithis', 'the', 'read', 'mode', 'this', 'the', 'read', 'mode', 'morning',
'batch', 'morning', 'batch', 'morning', 'batch', 'morning', 'batch']
len(res.split())
```

Assignment:

#wap to extract unique words from the text file.(use split,set)
#wap to create a dictionary consiste of words with its count from a given
  text file
out={'append':3(count word).....}
Note:(use,split,for loop,count())

Working with respect to csv File:

Q.what is csv file:
whenever we have a file with extension called.csv .theat file will be
conider as a csv file.csv refers--->comma separed value.
  csv-->comma separed values
  example: stname,sid,marks
        'pavi',101, 97
        'kavi',102, 85
        'davya',103,75

#Q.how can write the data into csv file?

```
#1.wrting the data into csv
import csv
with open('CSVFile1.csv','w')as file:
  #writer(): syntax: variable1=csv.writer(variable)
  b=csv.writer(file):it is used to give accessibility to write the data into csv file
  #1.writerow():it is used t insert single line row data
  #syntax: variable1.writerow(['col1','col2','col3'...'coln'])
  b.writerow(['s_name','sid','marks'])
  #2.writerows():it is used to insert multple rows
 syntax: variable1.writerow([['col1','col2','col3'...'coln'],
                        ['data1','data2','data3'....'datan'],
                        ['data1','data2','data3'....'datan'],
                        ['data1','data2','data3'....'datan']]....)
  b.writerows([['pavi','101',97],['kavi','102',80],['Davya','103',85]])
'''
```

```
#2.reading data from csv file:
import csv
with open('CSVFile1.csv','r') as file:
   #1.reader(): it is used to read the data from the csv file
   #syntax:variable1=csv.reader(variablename of csvfile store)
   b=csv.reader(file)
   b=[i for i in b if i!=[]]-->it is used to remove empty list in output
   b=[i for i in b if i!=[]][1:]--->to get only students data(rows expect
columnnames)
   print(list(b))
'''

#wap to create employe csv file and extract the details of who are getting
   morethan 25k salary
   #step1:import csv
   #step2:open()--w,mode
   #step3:writer()
   #step3:use write()
   #read the data eployee csv file
   step1:import csv
   step2:open()--->r,mode
   step3:after createing read that-->reader()
   Step4:remove empty in between by using forloop
   step5:codintion-->first traverse that data,if condition


#2.wap to extract details of employees whose name starts with 'A':
  #read the data eployee csv file
   step1:import csv
   step2:open()--->r,mode
   step3:after createing read that-->reader()
   Step4:remove empty in between by using forloop
   step5:codintion-->first traverse that data,if condition




import csv with open('EMP_FILE.csv','r') as file2:
b=csv.reader(file2)
b=[i for i in b if i!=[]]
b=[i for i in b if i!=[]][1:]
k=[j for i in b for j in i if type(i) in [list,tuple,set] and int(i[-1])>25000 ]
```

```
print(k)
```

Assignment

```
#wap to create employee csv file and extract the details of who are getting
  morethan 25k salary
  #step1:import csv
  #step2:open()--w,mode
  #step3:writer()
  #step3:use write()
  #read the data eployee csv file
  step1:import csv
  step2:open()--->r,mode
  step3:after createing read that-->reader()
  Step4:remove empty in between by using forloop
  step5:codintion-->first traverse that data,if condition


import csv
with open('EMP_FILE.csv','w') as file2:
  b=csv.writer(file2)
  b.writerow(['EMP_NAME','EMP_ID','EMP_SAL'])
  b.writerows([['AAAK','LO0012',28000],['JOHN','LO0014',30000],['clark','EMp
001',50000],['king','EMp002',18000],['Allen','LO0015','20000'],['AAAA','LO0015
','15000']])


import csv
with open('EMP_FILE.csv','r') as file2:
  b=csv.reader(file2)
  b=[i for i in b if i!=[]][1:]#--->if we want to extract only employee details
  print(list(b))
  for i in b:
    if i[2]>=str(25000):
      print(i)

#2.wap to extract details of employees whose name starts with 'A':
 #read the data eployee csv file
  step1:import csv
  step2:open()--->r,mode
  step3:after createing read that-->reader()
  Step4:remove empty in between by using forloop
```

step5:codintion-->first traverse that data,if condition

```
import csv
with open('EMP_FILE.csv','r') as file2:
  b=csv.reader(file2)
  b=[i for i in b if i!=[]][1:]#--->if we want to extract only employee details
  #print(list(b))
  for i in b:
    if i[0][0]=='A':
      print(i)


#Assignment:
#wap to extract unique words from the text file.(use split,set)

with open('SAMPLE1.txt','r') as file:
  res=file.read()
  words=res.split()
  print(len(words))
  print(set(words))
  print(len(set(words)))

#wap to create a dictionary consiste of words with its count from a given
  text file
out={'append':3(count word).....}
Note:(use,split,for loop,count())


with open('SAMPLE1.txt','r') as file:
  res=file.read()
  words=res.split()
  out={}
  for i in words:
    out[i]=words.count(i)
  print(out)

#wb-->Write Binary
with open('FirstFile.bin','wb') as f:
  f.write(b'Hello binary code')
```

```
with open('FirstFile.bin','wb+') as f:
  f.write(b'Hello binary code')

  f.write(b'x56xr5454')
  f.seek(0)
  print(f.read())

with open('FirstFile.bin','rb+') as f:
  print(f.read())
  f.seek(0)
  f.write(b'Hai ,hello,how are you')
```

#provide security to your data

#parsing techniques:
  1.Json parsing#--javascript object notation
  it is used to covert our data into string.
  2.pickle parsing

```
  {'name':'value',
   'age':'30',
   'sal':'50000'}
```

Json:

  Syntax for encryption:
    import json
    variable=json.dumps(variablename of data)
  *why dumps?
    it is used to convert the data into string format.it not available directly,we are going extract the function from module name called 'json'.

  syntax for decryption
    import json
    variable=json.loads(string data)

```
with open('FirstFile.bin','rb') as f:
```

```
  print(f.read())



#dumps()

import json
data=[10,20,30,40]
enc_data=json.dumps(data)
#print(data,type(data))#before converting into string format
#print(enc_data,type(enc_data))


with open('FILE2.txt','w') as a:
  a.write(enc_data)


#loads()

import json
with open('FILE2.txt','r') as a:
  res=a.read()
  data=json.loads(res)
  print(type(data),data)
```

Run the program in visual studio :
#encryption:

```
import json
#pip install cryptography
from cryptography.fernet import Fernet
key=Fernet.generate_key()
formatkey=Fernet(key)

data=[10,20,30,40]
convert=json.dumps(data).encode()
#print(data,type(data))#list
#print(convert,type(convert))#str
encrypted=formatkey.encrypt(convert)
```

```
withopen('FILE.json','wb') as f:
    f.write(encrypted)
#print('encrypted data saved:',encrypted)
#print('secret key:',key)
```

Dcryption:

```
#secret key: b'Mlj0uyauXeTUhL6hmcGYQy91Frjuj3QDTikt5pWngT0='
import json
from cryptography.fernet import Fernet
secret_key=b'Mlj0uyauXeTUhL6hmcGYQy91Frjuj3QDTikt5pWngT0='
format_key=Fernet(secret_key)
withopen('FILE.json','rb') as f:
    encrypted=f.read()
decrypted=format_key.decrypt(encrypted)
data=json.loads(decrypted)
print('decrypted data:',data)
print(type(data))
print(format_key)

#explanation:
```

import json
# pip install cryptography
from cryptography.fernet import Fernet
json → Used to convert Python objects (like list, dict) into a JSON string and back.

Fernet → A symmetric encryption tool from the cryptography library. It uses the same key for encryption and decryption.

key = Fernet.generate_key()
formatkey = Fernet(key)
Fernet.generate_key() → Creates a secret key (random bytes).

Fernet(key) → Creates a cipher object (formatkey) that can encrypt and decrypt data using this key.

data = [10, 20, 30, 40]
convert = json.dumps(data).encode()
data is a Python list.

json.dumps(data) → Converts the list into a JSON string: "[10, 20, 30, 40]".

.encode() → Converts the JSON string into bytes (because encryption works on bytes).

encrypted = formatkey.encrypt(convert)
Encrypts the JSON bytes.

Now encrypted is unreadable random text (ciphertext).

```
with open('FILE.json','wb') as f:
   f.write(encrypted)
```
Saves the encrypted data into a file FILE.json.

Mode 'wb' = write binary, because encryption produces binary data.

```
# decryption
secret_key = b'Mlj0uyauXeTUhL6hmcGYQy91Frjuj3QDTikt5pWngT0='
format_key = Fernet(secret_key)
```
A secret key is given explicitly (instead of the generated one earlier).

This key must be exactly the same as the one used to encrypt the file. Otherwise, decryption will fail.

```
with open('FILE.json','rb') as f:
   encrypted = f.read()
```
Reads the encrypted data back from the file.

```
decrypted = format_key.decrypt(encrypted)
data = json.loads(decrypted)
```
decrypt() → Converts ciphertext back into the original JSON bytes.

json.loads() → Converts JSON string back into a Python object (list).

```
print('decrypted data:', data)
print(type(data))
print(format_key)
```

Output:

decrypted data: [10, 20, 30, 40]
<class 'list'>

Key Points :
JSON is used to store structured data like lists/dictionaries in a text format.

Fernet encryption ensures the data is safe (cannot be read without the secret key).

The same key must be used for both encryption and decryption.

Files are written in binary mode (wb, rb) when working with encrypted data.

2.pickle parsing

#encapsulation
syntax: import pickle
variable=pickle.dumps(variablename of stored data)

why pickle.dumps()?
is used to convert our data into binary data

syntax:import pickle
variable=pickle.loads(binary data)

Why pickle.loads()?
it is used to convert binary data into origanal data

```
import pickle
data=[10,20,30,40,50]
enc_data=pickle.dumps(data)
#print(data)
#print(enc_data)
with open('FILE3.txt','wb') as f:
```

```
  f.write(enc_data)


#dacapsulation
import pickle
with open('FILE3.txt','rb') as f:
  b=f.read()
  print(b)
  data=pickle.loads(b)
  print(data)
'''
#wb-->write binary data ,wb+--->write and read binary
#rb-->read the binary data,rb+--->read the binary and and write binary
#ab--->write(add) binary data,ab+--->add and read the binary data
```

Saturday, September 13, 2025 7:43 AM